

Código: 98H04-06
POD - Programação Orientada a Dados



Python

Prof. Dr. Dalvan Griebler

Escola Politécnica,
Ciência de Dados e Inteligência Artificial (CDIA)

2026

Programação Orientada a Objetos

Classes

- Motivação
 - POO - Programação Orientada a Objetos (herança, polimorfismo, etc.)
 - Encapsular dados (dados ficam escondidos)
 - Gerenciamento de estado do dados (acessíveis ou não)
 - Reusar código
 - Organizar códigos da aplicações
 - Definir muito bem os papéis, ações e tipos de dados
 - No python qualquer coisa é considerado um objeto
 - Inteiros são da classe **int**
 - Números reais são da classe **float**

Classes

- Usamos classes?
 - Como uma template para criar instâncias de uma classe
 - Para definir métodos ou comportamento comum para as instâncias da classe
 - Para definir atributos ou campos para manipular dados dentro das instâncias
 - Para informar ou enviar mensagens
- Para que servem as classes?
 - Para criar objetos que:
 - São uma instância da classe
 - Tem seus próprios dados
 - Envia mensagem ou dados
 - Executam métodos das instâncias
 - Podem ter cópias com dados diferentes

Classes

- Definição

class palavra reservada para definir uma classe

() Pode ter nenhum ou mais parâmetros

: demarca o fim do cabeçalho da classe

__init__ função chamada toda vez que a classe é instanciada e usada para inicializar os dados

del palavra reservada para deletar o objeto

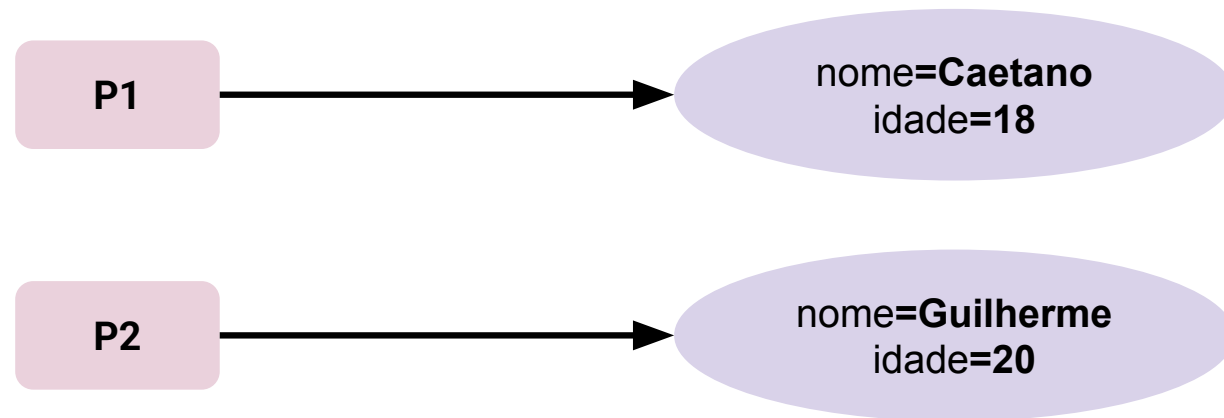
```
class nome_classe ():  
    __init__  
    atributos  
    metodos  
nome_objeto = nome_classe()  
del nome_objeto
```

Classes

- Criando objetos

```
P1=Pessoa("Caetano", 18)  
P2=Pessoa("Guilherme", 20)
```

Pessoa
nome idade



Classes

- Pode-se documentar no início da class **"""descrição da classe"""**
- **self** é uma variável especial e uma referência para a instância da classe.
 - É usada para acessar as variáveis que pertencem a classe
 - Pode ser qualquer outro nome, desde que seja o primeiro parâmetro da função **__init__**

```
class Pessoa:  
    def __init__(self, nome, idade):  
        self.nome = nome  
        self.idade = idade
```

```
P1=Pessoa("Caetano", 18)  
P2=Pessoa("Guilherme", 20)  
print(P1.nome,P1.idade)  
print(P2.nome,P2.idade)
```

Atributos da Classe

Criando Objetos

Acessando os Atributos

Classes

- As classes podem ter **suas variáveis** e são compartilhadas entre os objetos
 - Útil para contar a quantidade de instâncias da classe que foram criadas

```
class Pessoa:
    conta=0
    def __init__(self, nome, idade):
        Pessoa.conta+=1
        self.nome = nome
        self.idade = idade
P1=Pessoa("Caetano", 18)
P2=Pessoa("Guilherme", 20)
print(P1.nome,P1.idade)
print(P2.nome,P2.idade)
print(Pessoa.conta)
```

Classes

- As classes podem ter **seus métodos** para definir o comportamento delas
 - Identificado com um decorador
 - **@classmethod**
 - Primeiro argumento do método é a própria classe
 - Devem ser usadas apenas para:
 - Criar uma instância
 - Obter informações da classe
 - Gerenciamento de instâncias
 - Exemplos
 - Teste
 - Suporte

Representa a própria Classe

```
class Pessoa:
    conta=0
    @classmethod
    def incre_inst(cls):
        cls.conta+=1
    def __init__(self, nome, idade):
        Pessoa.incre_inst()
        self.nome = nome
        self.idade = idade
P1=Pessoa("Caetano", 18)
P2=Pessoa("Guilherme", 20)
print(P1.nome,P1.idade)
print(P2.nome,P2.idade)
print(Pessoa.conta)
```

Classes

- As classes podem ter **métodos estáticos** (funções livres)
 - Identificado com um decorador
 - **@staticmethod**
 - Não tem relação com a classe ou instância
 - São usadas apenas para agrupar funções ou organização

```
class Pessoa:
    conta=0
    @classmethod
    def incre_inst(cls):
        cls.conta+=1

    def __init__(self, nome, idade):
        Pessoa.incre_inst()
        self.nome = nome
        self.idade = idade

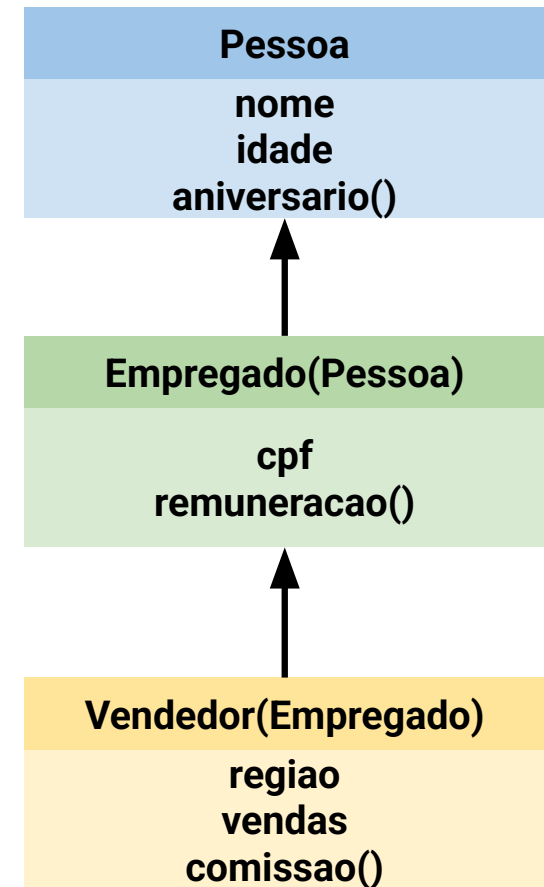
    @staticmethod
    def funcao_estatica():
        print("Func Estatica")

P1=Pessoa("Caetano", 18)
P2=Pessoa("Guilherme", 20)
print(P1.nome,P1.idade)
print(P2.nome,P2.idade)
print(Pessoa.conta)
Pessoa.funcao_estatica()
```

Classes

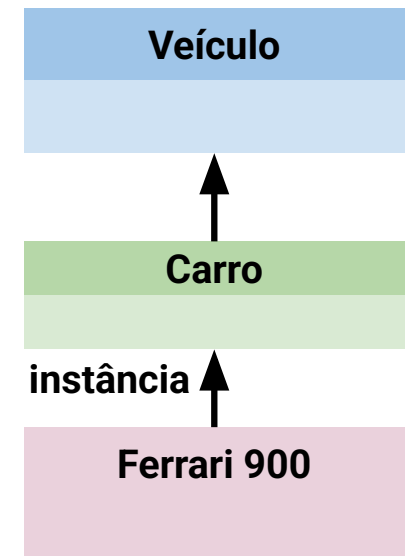
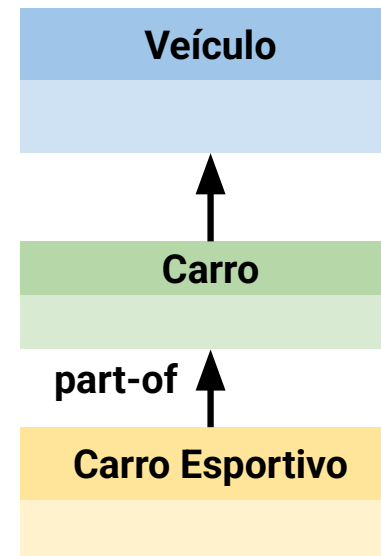
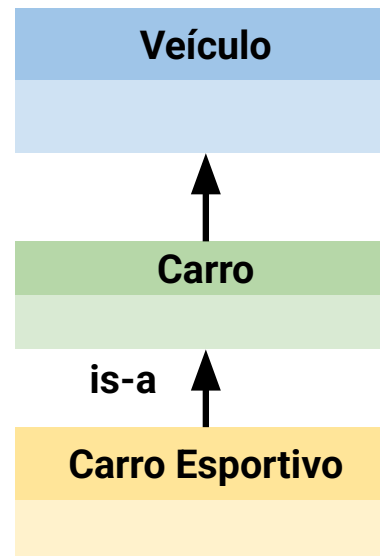
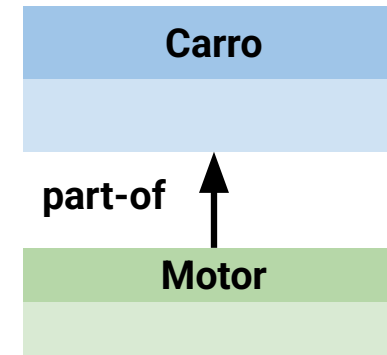
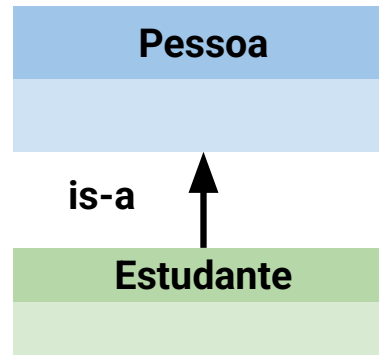
- **Terminologia e Herança**

- Classe:
 - combinação de dados e procedimentos nos dados
- Subclasse:
 - Uma classe que herda de outra classe
 - Uma classe pode ter várias subclasses
- Superclasse:
 - É a classe pai
- Herança única ou múltipla
 - No **Java** é herança única (uma classe)
 - No **Python**, é herança múltipla (de várias classes)



Classes

- Hierarquia
 - is-a (é um)
 - part-of (parte de)
 - Instância



Classes

- Podemos herdar os atributos e métodos de classes pai (superclasses)
 - Basta passar como parâmetro da nova classe
 - Se não quiser mudar nada, basta usar a palavra reservada **pass**
 - Também podemos estender/sobrescrever o método **__init__**
 - Também podemos herdar todos os métodos e atributos usando **super()**

```
class Estudante(Pessoa):  
    pass
```

```
class Estudante(Pessoa):  
    def __init__(self, nome, idade):  
        Pessoa.__init__(self, nome, idade)
```

```
class Estudante(Pessoa):  
    def __init__(self, nome, idade):  
        # novas propriedades
```

```
class Estudante(Pessoa):  
    def __init__(self, nome, idade, ano):  
        super().__init__(nome, idade)  
        self.ano_ingresso = ano
```

```
Estudante("Caetano", 18, 2021)
```

Classes

- Acessibilidade de Atributos e Membros (convenção de nomes)

- Públicos

- Acessível de qualquer lugar (dentro e fora)
 - Esse é o comportamento padrão
 - Usa-se sem prefixos

```
class Estudante(Pessoa):  
    universidade="PUCRS"
```

- Protegidos

- Somente acessível dentro da Classe e Subclasses
 - Usa-se o prefixo `_`

```
class Estudante(Pessoa):  
    _universidade="PUCRS"
```

- Privados

- Não é permitido acessar fora da Classe e Subclasses
 - Usa-se o prefixo `__`

```
class Estudante(Pessoa):  
    __universidade="PUCRS"
```

Classes

- Objetivos das Subclasses
 - Alterar o protocolo ou a interface
 - Extender os métodos ou atributos fornecidos pela classe
 - Alterar a implementação dos métodos
 - Adicionar outros comportamentos

Exercício de Fixação: Conta Bancária

- Defina a classe **ContaBancaria**
- Ao instanciar a classe, forneça:
 - número da conta
 - nome da pessoa
 - saldo inicial (em reais)
 - tipo de conta (corrente, poupança ou investimento)
- Fornecer três métodos de instância para esta classe
 - **deposito(quantidade)** para efetuar o depósito
 - **saque(quantidade)** para efetuar o saque
 - **saldo()** para obter o saldo da conta
- Monte um teste em que você realize todas as operações bancárias

Exercício de Fixação: Conta Bancária (Cont-1)

- Permita o rastreamento do número de instâncias criadas da classe **ContaBancaria**
- Mostre na tela uma mensagem toda vez que uma instância da classe é criada
- Mostre na tela a quantidade de contas criadas

Exercício de Fixação: Conta Bancária (Cont-2)

- O objetivo é estender a classe `ContaBancaria` fornecendo três Subclasses (**`ContaPoupanca`**, **`ContaCorrente`**, **`ContaInvestimento`**)
 - **`ContaCorrente`** precisa ter um limite de saque e redefinir o método para sacar do dinheiro. Este método precisa tratar se o saque for maior que o saldo.
 - **`ContaPoupanca`** precisa ter uma taxa de juros/rendimento associado
 - **`ContaInvestimento`** precisa ter um atributo que identifica o tipo de investimento (ex. baixo risco e alto risco)
 - Testar o funcionamento

Classes

- Sobrecarga de Operadores

- Uma maneira de definir classes para usar operadores aritméticos e lógicos
- Não é suportado no **Java**
- Outras linguagens como **C++**, **C#** e **Lua** também suportam sobrecarga de operadores
- Para poder implementar em **Python**, usa-se métodos especiais (__)
 - EX: `__add__()`, `__sub__()`, `__mul__()`

Sem sobrecarga de
operador

```
Q1 = Quantidade(5)
Q2 = Quantidade(10)
Q3 = Q1.add(Q2)
```

Com sobrecarga de
operador

```
Q1 = Quantidade(5)
Q2 = Quantidade(10)
Q3 = Q1 + Q2
```

Classes

- Sobrecarga de Operadores (operadores numéricos)

Operador	Expressão	Método Especial
Adição	$Q1 + Q2$	<code>__add__(self, Q2)</code>
Subtração	$Q1 - Q2$	<code>__sub__(self, Q2)</code>
Multiplicação	$Q1 * Q2$	<code>__mul__(self, Q2)</code>
Potência	$Q1 ** Q2$	<code>__pow__(self, Q2)</code>
Divisão	$Q1 / Q2$	<code>__truediv__(self, Q2)</code>
Divisão Inteira	$Q1 // Q2$	<code>__floordiv__(self, Q2)</code>
Resto da Divisão	$Q1 \% Q2$	<code>__mod__(self, Q2)</code>
-	$-Q1$	<code>__neg__()</code>
+	$+Q1$	<code>__pos__()</code>

Classes

- Sobrecarga de Operadores (operadores de comparação)

Operador	Expressão	Método Especial
Igual	$Q1 == Q2$	<code>__eq__(Q1, Q2)</code>
Diferente	$Q1 != Q2$	<code>__ne__(Q1, Q2)</code>
Menor que	$Q1 < Q2$	<code>__lt__(Q1, Q2)</code>
Menor que ou igual a	$Q1 \leq Q2$	<code>__le__(Q1, Q2)</code>
Maior que	$Q1 > Q2$	<code>__gt__(Q1, Q2)</code>
Maior que ou igual a	$Q1 \geq Q2$	<code>__ge__(Q1, Q2)</code>

Classes

- Sobrecarga de Operadores (operadores bitwise)

Operador	Expressão	Método Especial
AND (bits em ambos)	$Q1 \& Q2$	<code>__and__(Q1, Q2)</code>
OR (bit em qualquer)	$Q1 Q2$	<code>__or__(Q1, Q2)</code>
XOR (bits em um)	$Q1 \wedge Q2$	<code>__xor__(Q1, Q2)</code>
Deslocar bits a esquerda	$Q1 \ll \text{bits}$	<code>__lshift__(self, bits)</code>
Deslocar bits a direita	$Q1 \gg \text{bits}$	<code>__rshift__(self, bits)</code>
NOT (inverter bits)	$\sim Q1$	<code>__invert__()</code>

Classes

- Sobrecarga de Operadores (operadores com atribuição)

Operador com atribuição	Expressão	Método Especial
Adição	$Q1 += Q2$	<code>__iadd__(self, Q2)</code>
Subtração	$Q1 -= Q2$	<code>__isub__(self, Q2)</code>
Multiplicação	$Q1 *= Q2$	<code>__imul__(self, Q2)</code>
Potência	$Q1 **= Q2$	<code>__ipow__(self, Q2)</code>
Divisão	$Q1 /= Q2$	<code>__idiv__(self, Q2)</code>
Divisão Inteira	$Q1 //= Q2$	<code>__ifloordiv__(self, Q2)</code>
Resto da Divisão	$Q1 \%= Q2$	<code>__imod__(self, Q2)</code>
AND	$Q1 \&= Q2$	<code>__iand__(Q1, Q2)</code>
OR	$Q1 = Q2$	<code>__ior__(Q1, Q2)</code>
XOR	$Q1 \^{}= Q2$	<code>__ixor__(Q1, Q2)</code>
Deslocar bits a esquerda	$Q1 <<= Q2$	<code>__ilshift__(self, bits)</code>
Deslocar bits a direita	$Q1 >>= Q2$	<code>__irshift__(self, bits)</code>

Classes

- Sobrecarga de Operadores (outros operadores unários)
 - `__abs__` Número absoluto de um objeto
 - `__complex__` Números complexos
 - `__int__` Números inteiros
 - `__long__` Números inteiros longos
 - `__float__` Números exatos
 - `__oct__` Números na base octal
 - `__hex__` Números na base hexadecimal (números e símbolos)
 - `__bool__` Valores booleanos

Exercícios de Fixação: Sobrecarga de Operadores

- Crie uma classe chamada **Distancia**, que terá implementado a sobrecarga de operadores para:
 - Adição
 - Subtração
 - Multiplicação
 - Potência
 - Divisão
 - Divisão inteira
 - Resto da Divisão
- Suporte para mostrar o resultado como uma string
- Suporte que estas operações possam ser feitas com números da classe **int** e **float**
- Suporte a operação com atribuição

Classes

- Propriedades

- POO foca em esconder os dados dentro de objetos
- POO objetiva fornecer métodos para acessar/manipular os dados dos atributos
- Em python, o conceito de encapsulamento não é explícito
 - Usa convenções de nomes (`_` ou `__`), que nem sempre garantem ou protegem o acesso
 - Usa propriedades para definir **setters** e **getters**

```
class Pessoa:
    def __init__(self, nome, idade):
        self._nome = nome
        self._idade = idade
P1=Pessoa("Caetano", 18)
print(P1._nome,P1._idade)
P1._nome= "Caetano Muller"
P1._idade= "19"
print(P1._nome,P1._idade)
```



```
$ python codigo.py
Caetano 18
Caetano Muller 19
```

Classes

- Propriedades

- Embora precisamos de mais código para expressar os métodos de GETTER e SETTER:
 - Ajuda e facilita o tratamento de exceções para reportar erros
 - Deixa mais claro o que estamos fazendo nos dados
- Usando este estilo de trabalho em classes, também nos permite controlar acesso aos atributos
 - **Ex:** o atributo **_nome** ser somente leitura. Portanto, um método SETTER não é usado
- Sintaxe para definir uma propriedade:

```
Nome_propriedade = property (fget=None, fset=None, fdel=None, doc=None)
```

Classes

- Exemplo de uso de **property**

```
class Pessoa:
    def __init__(self, nome, idade):
        self._nome = nome
        self._idade = idade
    def get_idade(self):
        return self._idade
    def set_idade(self, nova_idade):
        if isinstance(nova_idade, int) & nova_idade > 0 & nova_idade < 120:
            self._idade = nova_idade
    def del_idade(self):
        del self.idade
    idade = property(get_idade, set_idade, del_idade, doc="Propriedade para o atributo idade")

P1=Pessoa("Caetano", 18)
P1.idade=19
print(P1.idade)
```

Classes

- Usando decoradores
 - São metadados que anotamos no código para o interpretador Python
 - Conhecido como metaprogramação (dicas ao interpretador para modificar os códigos)
 - São usados para definir o comportamento de uma classe ou função
 - Podemos usá-los para melhorar a forma de definir SETTER e GETTER

```
class nome:  
    @property  
    def func():  
  
    @func.setter  
    def func():  
  
    @func.deleter  
    def func():
```

Classes

- Exemplo de uso de decoradores SETTER e GETTER

```
class Pessoa:
    ...
    @property
    def idade(self):
        return self._idade
    @idade.setter
    def idade(self,nova_idade):
        if isinstance(nova_idade,int) & nova_idade > 0 & nova_idade < 120:
            self._idade = nova_idade
    @idade.deleter
    def idade(self):
        del self.idade

P1=Pessoa("Caetano", 18)
P1.idade=19
print(P1.idade)
```

Classes

- Métodos Especiais

- Python tem vários métodos especiais para classes e todos levam a mesma sintaxe
 - `__nomeMetodo__()`
- Alguns que usamos até agora
 - `__init__()`
 - `__str__()`

- Atributos Intrínsecos das classes

- `__name__` nome da classe
- `__doc__` descrição da classe (se tiver sido feita com aspas triplas)
- `__module__` módulo que a classe faz parte
- `__dict__` dicionário contendo todos os atributos e métodos da classe

- Atributos Intrínsecos dos objetos

- `__class__` nome da classe que o objeto foi criado
- `__dict__` dicionário com todos os atributos e seus valores de um objeto

Exercício de Fixação: Conta Bancária (Cont-3)

- Use decoradores na classe **ContaBancaria** para definir o acesso aos atributos com os métodos criados.
 - número da conta
 - nome da pessoa
 - saldo inicial (em reais)
 - tipo de conta (corrente, poupança ou investimento)

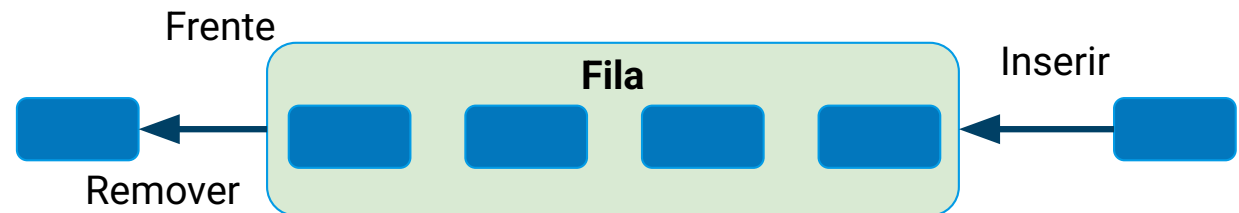
Tipos Abstratos

- Exemplos de tipos abstratos de dados são Fila e Pilha
- Fila e Pilha não são oferecidos em Python, mas podem ser implementados
- São modelos para definir um tipo de dados e seu comportamento
 - Valores
 - Operações nos dados
 - Comportamento das operações
- Ex. Listas precisam oferecer métodos e um comportamento adequado
- Filas se comportam como FIFO (First-In-First-Out)
- Pilhas se comportam como LIFO (Last-In-First-Out)
- Outros exemplos são:
 - listas encadeadas, grafos, árvores, etc.

Tipos Abstratos

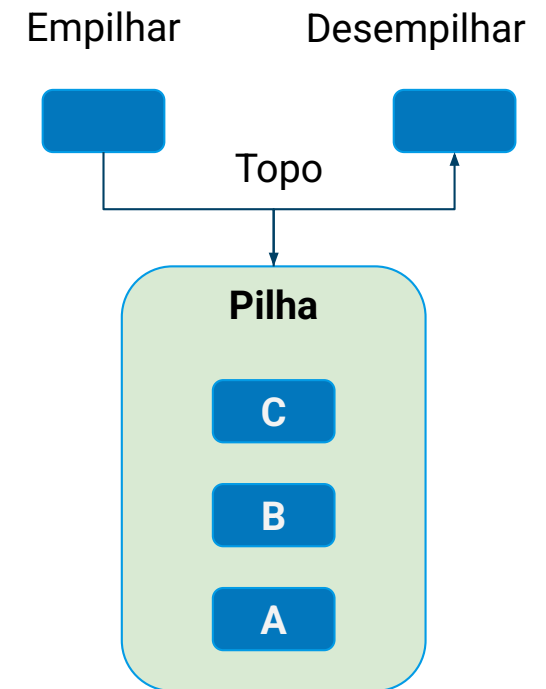
- Fila (FIFO)

- As operações básicas são:
 - Criação da fila
 - Adicionar elemento no fim da fila
 - Remover elemento na frente da fila
 - Determinar o tamanho da fila
 - Determinar se a fila está vazia
 - Podem ser de tamanho fixo ou variável
- Também é possível implementar prioridade para os elementos e acessá-los sem remover
- Podem ser implementadas usando Listas
 - **append()**
 - **pop(0)**



Tipos Abstratos

- Pilha (LIFO)
 - As operações básicas são:
 - Criação de pilha
 - Adicionar elemento no topo da pilha
 - Remover elemento do topo da pilha
 - Determinar o tamanho da pilha
 - Determinar se a pilha está vazia
 - Podem ser de tamanho fixo ou variado
 - Outro comportamento é permitir acesso ao elemento sem remover
 - Podem ser implementadas usando Listas
 - **append()**
 - **pop()**



Exercícios de Fixação sobre Tipos Abstratos

- Crie uma classe com comportamento de fila (FIFO)
 - **enqueue(elemento)** para adicionar elemento no fim da fila
 - **dequeue()** para remover do início da fila
 - **peek()** para retornar o elemento do início da fila sem remover
 - **__len__()** para retornar o tamanho da fila
 - **is_empty()** para verificar se a fila está vazia
 - **__str__()** para converter a fila em uma string
- Crie uma classe com comportamento de pilha (LIFO)
 - **push(elemento)** para adicionar elemento no topo da pilha
 - **pop()** para remover do topo da pilha
 - **top()** para retornar o elemento do topo sem remover
 - **__len__()** para retornar o tamanho da pilha
 - **is_empty()** para verificar se a pilha está vazia
 - **__str__()** para converter a pilha em uma string

Tratamento de Erros e Exceções

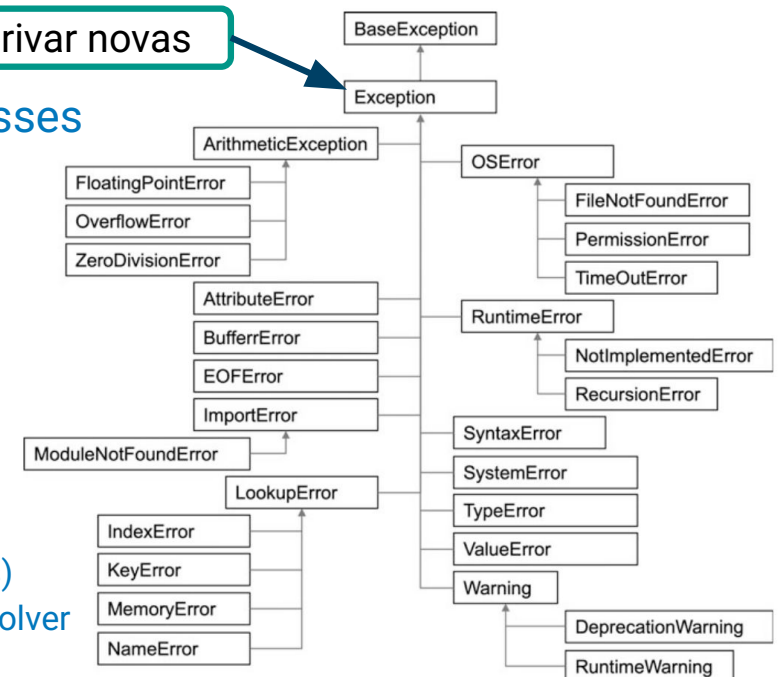
- Tipos de Erros
 - Sintaxe (uso incorreto da linguagem)
 - Lógicos (arquivo que não existe, operação indevida, dados incorretos, etc)
- O interpretador Python interrompe assim que se depara com um erro
- Alguns erros podem ser tratados e outros não
- Podemos também informar o usuário sobre o erro para que ele possa resolver
- Também podemos fazer um teste de “sanidade” em uma expressão
 - Útil para deixar o código mais confiável
 - Útil para alertar de problemas que não queremos aceitar e o usuário tem que resolver
 - Útil para depuração de código
 - Usa-se para testar uma expressão que não pode ser falsa
 - Não é recomendado usar para dados provenientes de fontes externas (usar exceções para isso)

Tratamento de Erros e Exceções

<https://docs.python.org/3/library/exceptions.html>

- Objetivo é lidar com erros em execução
- Erros e exceções em Python são tratadas com classes
- Conceitos:
 - **Exception**: erro gerado em execução
 - **Raising**: quando ocorre a exceção
 - **Throwing**: quando o código não consegue lidar
 - **Handling**: processamento que lida com o erro
 - **Handler**: código que lida com o erro (**catch**)
 - **Signal**: tipo específico de exceção
- Exceção são implementadas para
 - Erros de causas externas (hardware, rede, arquivo e usuários)
 - Quando se sabe que pode ter um erro e tem opções para resolver
 - Oferecer ao usuário informações do erro que são facilmente corrigidas

Derivar novas



Tratamento de Erros e Exceções

- Vocabulário para manipular erros e exceções
 - **raise** usado para lançar uma exceção de qualquer lugar
 - **assert** testar uma condição e se retornar False, lança uma exceção
 - **try** cláusula que executa até encontrar uma exceção
 - **except** cláusula para capturar e tratar as exceções da cláusula **try**
 - **else** cláusula que executa quando não tem exceções no **try**
 - **finally** cláusula que executa sempre depois do **try** (mesmo com exceções)

raise <Exception/tipo da exceção>

assert <condição> [, <mensagem>]

try:

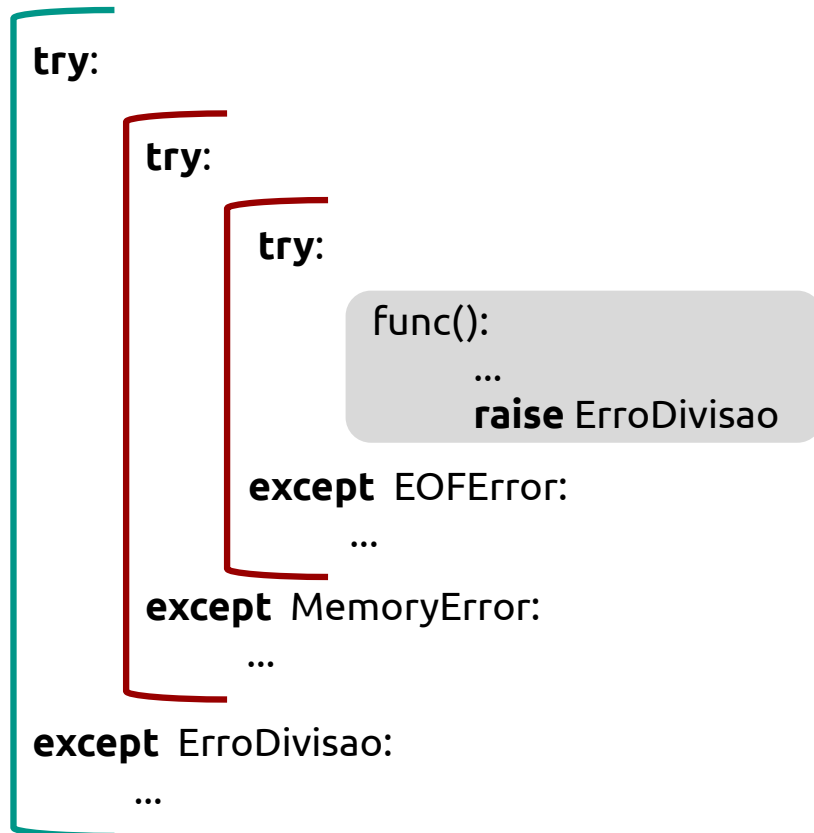
#bloco de código a ser monitorado

except <tipo da exceção>:

#bloco de código a ser chamado se uma exceção ocorre

Tratamento de Erros e Exceções

- Aninhamento de Exceções
 - Quando um erro acontece, ele pode passar adiante até chegar no caso em que é possível tratar o erro.
 - Isso é o que acontece no sistema



Exercício de Fixação: Conta Bancária (Cont-4)

- Adicionar tratamento de exceção na classe **ContaCorrente** implementada no exercício anterior, onde não é permitido o saque e depósito de valores negativos.
 - Defina uma nova classe para tratamento de exceção chamada **ErroQuantidade**, que deve receber como parâmetro a conta e a mensagem de erro.
 - Atualize também os métodos de depósito e saque na classe **ContaBancária** e **ContaCorrente** para que lancem a exceção **ErroQuantidade** se o valor fornecido for negativo.
 - Crie uma classe **ErroSaldo** para lidar com um possível saque abaixo ou acima do limite. É necessário mostrar a informação de qual conta gerou este erro.
 - Use blocos **try** e **except** para realizar os testes

Módulos e Pacotes

- Módulos (como códigos de uma biblioteca)
 - Agrupamento de classes, funções e códigos genéricos, que podem conter:
 - Funções
 - Classes
 - Variáveis
 - Código Executável
 - Atributos associados ao módulo, tal como o seu nome (nome do arquivo *.py).
 - Recomenda-se que os códigos sejam organizados em módulos quando ficam grandes
 - Dividir o código em módulos simplificam a compreensão, teste, reuso, etc.
 - Para acessar os códigos dos módulos, é necessário importar ele com a palavra **import**

```
import modulo1  
Import modulo2, modulo3, modulo3
```

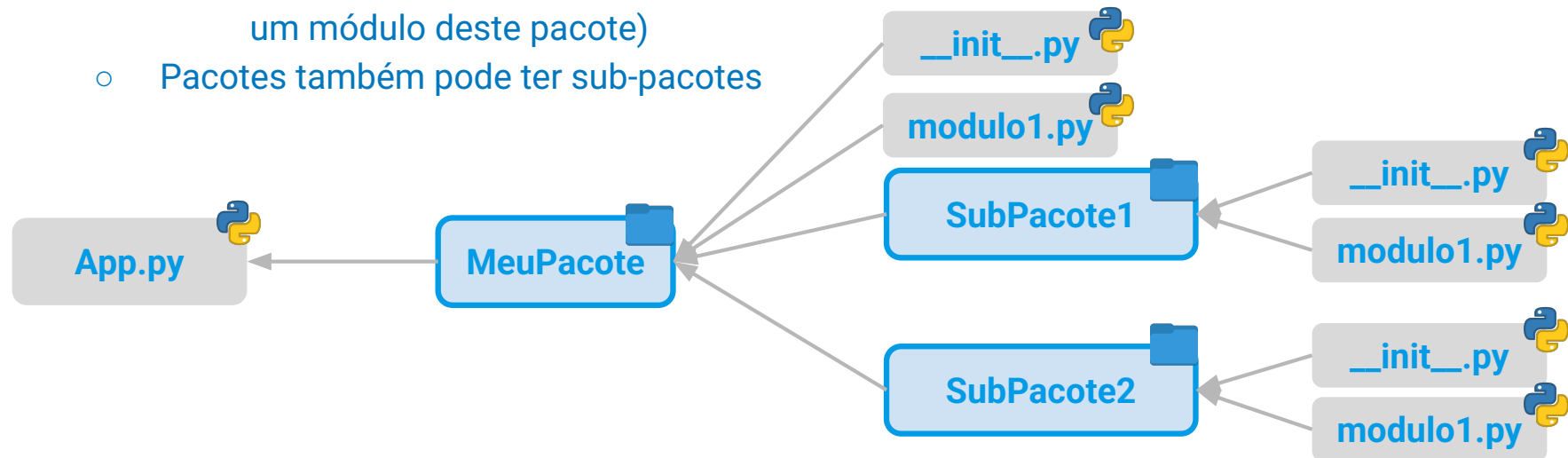
Módulos e Pacotes

- Módulos (como códigos de uma biblioteca)
 - Podemos importar coisas específicas de um módulo
 - É possível também criar Alias/Atalhos/Renomear
 - Dentro de um módulo, pode-se esconder elementos com `_` (underline)
 - É permitido importar módulos dentro de funções
 - Existem propriedade inerentes
 - `__name__`
 - `__doc__`
 - `__file__`
 - Existem módulos padrão (<https://docs.python.org/3/library/>)
 - `sys`
 - Além do **PIP**, também existe o **ANACONDA** para gerenciamento de módulos.
 - Módulos são também Scripts, ou seja, são executados.

```
from modulo1 import *  
from modulo1 import MinhaClasse  
from modulo1 import MinhaClasse as MC
```

Módulos e Pacotes

- Pacotes (organização hierárquica de módulos baseado em diretórios)
 - Pacotes em Python devem conter:
 - Um diretório com um ou mais arquivos
 - Um arquivo com o nome `__init__.py` (esse pode conter código executável ao importar um módulo deste pacote)
 - Pacotes também pode ter sub-pacotes



Módulos e Pacotes

- Argumentos da Linha de Comando
 - sys
 - getopt
 - argparse
 - optparse
- Variáveis Constantes
 - Variáveis que não alteram seu valor
 - Várias constantes estão presentes nas bibliotecas
 - Convenção recomendada
 - Colocar em um módulo
 - Usar letras em maiúsculas para o nome das variáveis

```
import constantes as const  
print(const.VAR)
```

Exercício de Fixação: Conta Bancária (Cont-5)

- O objetivo é criar um módulo para armazenar as classes criadas para o exercício Conta Bancária. Sendo assim:
 - Coloque as classes **ContaBancaria**, **ContaCorrente**, **ContaPoupanca**, **ErroQuantidade**, **ErroSaldo** em um arquivo chamado **contas.py**. Coloque este arquivo dentro de um pacote/diretório chamado **MeuBanco**.
 - Em um novo arquivo, no diretório raiz, importe o pacote **MeuBanco** e o módulo **contas** para efetuar operações de teste.
 - Faça um programa de teste, que recebe os dados para testar e inicializar as classes usando os argumentos da linha de comando. **Ex:**
 - `./teste.py <NumeroConta> <NomeIndividuo> ...`

Manipulação de Arquivos

- Abrir arquivos **open(nome-arquivo, MODO)**
 - Modos
 - r abrir para leitura apenas
 - r+ abre para leitura e escrita
 - a abrir para escrita ao final do arquivo e cria um arquivo se não existir
 - w abrir para escrita, apaga os dados e cria um arquivo se não existir
 - x abrir para escrita, cria um novo arquivo e se já existe ele falha
 - t abre como arquivo de texto (modo padrão, convertendo de bytes para unicode)
 - b abre como arquivo binário (ex. "rb" ou "wb")

```
F = open("arquivo.txt", "w")
```

Manipulação de Arquivos

- Métodos do objeto arquivo

- **close()** fechar o arquivo
- **flush()** limpar o buffer
- **fileno()** retorna o descritor do arquivo (um valor inteiro)
- **isatty()** retorna **True** se está conectado a um terminal interativo
- **readable()** retorna **True** se pode ser lido o arquivo
- **writable()** retorna **True** se pode ser escrito no arquivo
- **seekable()** retorna **True** se pode ser ter acesso aleatório no arquivo
- **read([tamanho-bytes])** lê todo o arquivo ou quantidade de bytes especificada
- **readline([tamanho-bytes])** lê toda a linha ou quantidade de bytes especificada da linha
- **readlines([tamanho-bytes])** lê várias linhas ou quantidade de bytes especificada
- **seek(posicao,[modo])** posiciona o cursor na posição informada (modo 0=absoluto, 1=atual, 2=EOF)
- **tell()** retorna a posição atual do curso
- **truncate([tamanho-bytes])** redimensionar o tamanho do arquivo
- **write(string)** escreve uma string no arquivo
- **writelines(sequencia)** escreve uma sequência de strings no arquivo
- **detach()** des-anexar o arquivo do objetivo

Manipulação de Arquivos

- Atributos do objeto arquivo (somente depois de aberto o arquivo)
 - **closed** retorna verdadeiro se o arquivo está fechado
 - **mode** retorna de que modo o arquivo foi aberto
 - **name** retorna o nome do arquivo aberto
- Usando **with** para abrir arquivo, não precisamos fechá-lo

```
with open("arquivo.txt", "r") as file:  
    print(file.read())
```

Manipulação de Arquivos

- Exemplo de Funcionamento

```
arq=open("Arquivo.txt", "r")
```

Arquivo.txt

Aula de POD!
Dalvan
Griebler

arq= A u l a \ d e \ P O D \n D a l v a n ... r \n



```
bytes=4  
arq.tell()  
arq.read(bytes)  
arq.tell()  
arq.seek(0,2)  
arq.tell()
```

Exercícios de Fixação sobre Arquivos

- Escreva um programa que:
 - Gera dois arquivos:
 - **1.txt** deve conter uma coluna de números de 1 até 20
 - **2.txt** deve conter uma coluna de números de -20 até -1
 - Junte o conteúdo dos dois arquivos e escreva em um arquivo **1+2.txt**
- Escreva um programa que:
 - Implementa a função **bytes_ate(padrao)** que retorna o valor em bytes até encontrar o seguinte padrão de linha **"-6 15\n"** no arquivo **1+2.txt**
 - Reduza o arquivo **1+2.txt** ao tamanho de bytes que a função **bytes_ate()** retornou
- Escreva um programa que:
 - Use a função **bytes_ate(padrao)** para ler as linhas a depois do padrão **"-15 6\n"** até o final do arquivo **1+2.txt**, mostrando-as na tela.
 - Mostre na tela quantos bytes do arquivo **1+2.txt** não foram mostrados na tela
- Implemente um programa que mostra a resolução (height x width) de uma imagem **.jpeg**

1+2.txt

```
-20 1
-19 2
-18 3
...
```

Classes Abstratas

- São classes que não podem ser instanciadas (não podemos criar objetos)
- Espera-se que elas sejam estendidas por uma ou mais subclasses
- São bastante úteis para implementar herança de classes com reuso
- As classes abstratas podem deixar métodos para serem implementados
- Podemos forçar que as subclasses implementem métodos específicos
- Quando usar classes abstratas?
 - Quando queremos definir dados e comportamentos comum para várias subclasses
 - Quando desejamos forçar subclasses a se comportar de uma forma específica
- No Python, elas são mais conhecidas como ABCs (*Abstract Base Classes*)

Classes Abstratas

- O que as ABCs podem ter?
 - Zero ou mais métodos abstratos ou propriedades (não é um requisito)
 - Zero ou mais métodos concretos e propriedades (não é um requisito)
 - Atributos protegidos e privados (lembrem-se que é usado `_` ou `__`)
- ABCs podem ser usadas para especificar interfaces e protocolos
- ABCs são bastante usadas dentro do python, ex:
 - Módulos: `collection`, `numbers`, `IO`, ...
- Interfaces são implícitas (contrato entre uma interface e o usuário da mesma)
 - ABCs são usados para estabelecer um contrato entre a classe e quem está usando ela
- Podemos registrar subclasses virtuais sem que elas possuem relação de herança
- Classes mistas são implícitas e são uma categoria de ABCs

Exercício de Fixação: Conta Bancária (Cont-6)

- O objetivo é implementar uma classe abstrata (ABC) na classe **ContaBancaria**, que tem sido uma classe concreta. Assim, a criação das contas vai se parecer da seguinte forma.

```
conta1 = ContaCorrente("343", "Ceatano", 20.50, 115.50)
conta2 = ContaPoupanca("355", "Morgana", 53.45, 1.5)
conta3 = ContaInvestimento("357", "Guilherme", 82.55, "Risco Alto")
```

Protocolos, Polimorfismo e Descritores

- Antes de tudo, é preciso sempre ter em mente que contratos são implícitos
 - No Java, existe o conceito de **interface** para implementar os contratos
 - No Python, por exemplo, uma classe que implementa um método de **soma** o contrato é:
 - O método **soma** vai suportar qualquer coisa que possa ser somado numericamente.
- Python tem o conceito de *Duck Typing* (tipagem estrutural) que vem de:
 - ***If it walks like a duck, swims like a duck and quacks like a duck then it's a Duck!***
 - Ex: se o tipo pode ser usado com soma, multiplicação, subtração e divisão, logo isso pode ser tratado como um tipo numérico, mesmo que não for.
 - Em alguns casos, nem todos os contratos precisam ser implementados se você não usá-los
 - Através deste conceito, que os contratos são implicitamente estabelecidos entre as classes
- Python não tem um mecanismo formal para estabelecer o que é necessário entre o fornecedor e o usuário de uma funcionalidade

Protocolos, Polimorfismo e Descritores

- Então, como sabemos o que é necessário?
 - No Python, usa-se o conceito de protocolo
 - Trata-se de uma descrição informal do programador
 - Isso é feito através de documentação ou funções solitárias (string de documentação da classe)
- Exemplos de protocolos
 - Sequencias como containers (acesso de itens em forma de lista)
 - Requer que sejam implementados os métodos `__len__()` e `__getitem__()`
 - Gerenciamento de Contexto
 - Requer que sejam implementados os métodos `__enter__()` e `__exit__()`
- É através de Protocolos que o conceito Polimorfismo de POO opera em Python

Protocolos, Polimorfismo e Descritores

- Polimorfismo

- Habilidade de enviar uma mesma mensagem para diferentes objetos, parecendo que todos estão chamando a mesma função
- Essencialmente, buscamos fazer com que a mesma operação seja realizada por diferentes coisas.
- A forma com que isso é feito, vai depender das coisas que estão recebendo esta mensagem ou requisição. Porém, o programador não se preocupa em lidar com elas.
- **Ex:** Superclasse e subclasse possuem os mesmos nomes de métodos e o programa sabe quais métodos devem ser chamados. Consequentemente, os objetos criados a partir destas classes assumem diferentes formas.

Protocolos, Polimorfismo e Descritores

- Descritores

- É um protocolo para gerenciamento de atributos dos objetos
- Este protocolo protege o acesso direto ao atributo
- O protocolo é implementado através dos seguintes métodos especiais:
 - **__get__()** método é chamado quando um valor de um atributo é acessado
 - **__set__()** método é chamado quando um valor de um atributo é atribuído/setado
 - **__delete__()** método é chamado quando deletamos o atributo
 - **__set_name__()** método chamado no momento em que dono da classe é criado
- As classes que implementam o protocolo Descritor só podem ser instanciadas dentro de outra classe (classe que desejamos monitorar os atributos)

Exercício de Fixação: Conta Bancária (Cont-7)

- O objetivo é implementar o protocolo de gerenciamento de contexto.
 - Modifique a classe **ContaBancaria** para permitir este protocolo
 - Note que estes métodos especiais implementados serão herdados pelas subclasses **ContaCorrente**, **ContaInvestimento**, **ContaPoupanca**.
 - Exemplo de teste:

```
with ContaCorrente("343", "Ceatano", 20.50, 115.50) as c:  
    c.deposito(20.30)  
    c.saque(15.20)  
    print(c.saldo)
```

Modificação Dinâmica de Atributos

- Esse conceito é conhecido como **Monkey Patching**
- O objetivo é realizar mudança, incluir novas funcionalidade ou correções em tempo de execução
- Em Python, isso é muito mais fácil fazer porque não existe uma proteção estrita em variáveis
- Em teoria, para que usamos?
 - Substituir métodos, classes, atributos e funções
 - Modificar o comportamento de módulos sem realizar uma cópia
 - Realizar alteração diretamente na memória
 - Permitir correções de bugs e problemas de segurança em tempo de execução
 - Automatizar e se recuperar de falhas

Modificação Dinâmica de Atributos

- Exemplos práticos em Python

- Esquecer de implementar um método de um protocolo e precisar usá-lo mais tarde.
- Adicionar novos atributos a uma classe já implementada
- Lidar com acesso a atributos e métodos desconhecidos
- Interceptar a pesquisa e o acesso aos dados dos atributos
- Interceptar a modificação dos dados nos atributos

```
print(Classe.__dict__)\nprint(Objeto.__dict__)
```

- Funcionamento e manipulação de atributos em Python

- Atributos e métodos de um objeto ou da classe são armazenados em dicionários **__dict__** (chave-valor).
- Toda vez que um atributo ou método é acessado ou modificado com a sintaxe padrão usando ponto, uma busca é realizada no **__dict__**.
- A busca não é realizada se acessamos diretamente **__dict__**.

```
print(Classe.Y)\nprint(Objeto.X)
```

Exercício de Fixação: Conta Bancária (Cont-8)

- Implemente na classe **ContaBancaria** o acesso a métodos e atributos desconhecidos, mostrando uma mensagem na tela e retornando o valor **-1**.
 - Exemplo de acesso a um atributo e método desconhecido:

```
print("conta1.teste", conta1.teste)  
print("conta1.teste()", conta1.teste())
```

- Saída esperada:

```
__getattr__: atributo desconhecido -> teste  
conta1.teste: -1  
__getattr__: atributo desconhecido -> teste()  
conta1.teste(): -1
```

Iterável, Iteradores, Geradores e Co-rotinas

- Protocolo Iterável
 - Usado para processar tipos de dados um a um
 - O iterável é usado por um iterador para realizar o processamento
 - Exemplos de tipos iterável: Listas, Conjuntos, Dicionários, Range e Tuplas
 - Esses tipos são container iteráveis para ser usado por um iterador.
 - Para suportar este protocolo, a classe precisa implementar o método `__iter__()`
 - Espera-se que ele forneça uma referência ao objeto iterador, que pode ser o seu ou outro tipo de dado qualquer
- Protocolo Iterador
 - Trata-se de um objeto que vai retornar uma sequência de valores.
 - Podem ser finitos ou infinitos
 - Para suportar este protocolo, a classe precisa implementar o método `__next__()`
 - Espera-se que o método retorne o próximo item de uma sequência ou lançar a exceção **StopIteration** para indicar que o iterador concluiu.
- Dica: módulo **itertools** fornece várias funcionalidade para iteradores

Iterável, Iteradores, Geradores e Co-rotinas

- Geradores (produtores de dados)
 - São funções especiais que geram uma sequência de valores para serem iterados sob demanda (ou seja, quando precisar) ao invés de serem produzidos anteriormente.
 - Funções deste tipo possuem declarações com a palavra reservada **yield**
 - **yield** executa uma vez para cada valor especificado
 - A cada ciclo de um laço **for**, um novo valor é obtido
 - Essa função vai fornecer valores até que ela retorne algo
- Co-rotinas (consumidores de dados)
 - São funções cujo o objetivo é consumir dados
 - **send()** é usado para enviar dados para uma co-rotina
 - **yield** é usado para aguardar até que um dados seja enviado para a co-rotina
 - **next()** é usado para preparar uma co-rotina a ser usada
 - **close()** é usado para encerrar uma co-rotina

Exercícios de Fixação sobre Geradores

- Implemente um gerador de números primos que precisa ter um limite **gerador_numero_primo(limite)**.
- Depois, crie um gerador de números primos infinito **gerador_numero_primo_infinito()**, sendo capaz de gerar números primos até que não seja mais usado. Teste usando **next()**.

Referências

1. HUNT, JOHN . A Beginners Guide to Python 3. Springer , 2019.
2. PHILLIPS, DUSTY . Python 3 Object Oriented Programming. (3th Edition)
PACKT , 2018.
3. Biblioteca padrão Python 3: <https://docs.python.org/3/library/>

Agradecimentos

- Prof. Felipe Meneguzzi
 - Discussão e exercícios de programação imperativa
- Prof. Julio Machado
 - Discussão e dicas sobre programação funcional
- Prof. Isabel Manssour
 - Discussão e dicas sobre programação orientada a objetos
- Prof. Daniel Callegari
 - Discussão e direções sobre condução da disciplina
- Prof. Fernando Pereira
 - Conteúdo sobre programação funcional